

NR2DT-99: Best Practice Summary

Series: NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 10 of 10 |
Created: April 2026 | **Last Updated:** 10/05/2026

Overview

Reference card for everything the NR2DT series prescribes. Use this as the post-mortem checklist and as the ongoing reference for similar migrations.

Table of Contents

- 1. [The 9-Step Framework](#)
- 2. [Best Practices by Step](#)
- 3. [Common Pitfalls](#)
- 4. [Where the Deep Dives Live](#)
- 5. [When to Reuse This Series](#)

Prerequisites

Requirement	Details
Audience	Migration lead writing the post-mortem; future teams reusing the framework
Format	Reference summary — best-practice card, no procedural commands

1. The 9-Step Framework

Step	Notebook	Output
1	NR2DT-01 Discover	Inventory + translation report + effort estimate
2	NR2DT-02 Strategize	Wave plan + scope decisions + gate definitions
3	NR2DT-03 Design	Bucket + host group + IAM + OpenPipeline design
4	NR2DT-04 Translate	All NRQL converted to DQL; LOW triaged
5	NR2DT-05 Migrate Dashboards & Alerts	Wave 0/1/3 imported; dual-alert window completed
6	NR2DT-06 Migrate Synthetics, SLOs & Workloads	Wave 2/4 imported; SLO 7-day delta validated
7	NR2DT-07 Migrate Logs, Tags & Drops	Wave 5 imported; log forwarders reconfigured
8	NR2DT-08 Validate	Three-tier validation passed
9	NR2DT-09 Cutover, Rollback & Decommission	Cutover complete; NR archived
99	NR2DT-99 Summary	This reference card

2. Best Practices by Step

#	Practice	Why It Matters
1	Treat discovery as a deliverable, not a task	Skipping it makes every later step a guess
1	Run the translator early to size the LOW pile	LOW is the dominant cost driver
2	Get stakeholder sign-off on the <i>plan</i> , not just delivery	Surfaces scope disputes early

#	Practice	Why It Matters
2	Define gate criteria before each wave starts	Prevents "are we done?" debates
3	Lock in bucket strategy + IAM model before any artifact migrates	Both are foundational; retrofit is painful
4	Triage every LOW; do not let auto-translation hide them	Wrong DQL silently produces wrong data
5	Dashboards first (low risk) builds trust in the translation pipeline	Catches systemic issues before alerts
5	Dual-alert for 1–2 weeks before silencing NR	Catches volume / detection delta
5	Promote alerts policy-by-policy, not in batch	Reversible per-policy
6	Measure the 7-day NR-vs-DT SLI delta (by hand) and run <code>audit-slos</code> before declaring SLO migration done	The delta catches math drift; <code>audit-slos</code> catches invalid SLO metrics
7	Dual-ship logs during cutover; volume parity validates the switch	Ensures no log gap
8	All three validation tiers must pass; skipping Tier 3 is the most common error	Syntactically valid ≠ semantically correct
9	Retain rollback manifests ≥ 30 days post-cutover	Late-discovered regressions still recoverable
9	Halt NR ingest after dashboards archived, not before	Preserves dual-source comparison capability

3. Common Pitfalls

Pitfall	Symptom	Mitigation
Skipping discovery	"How many dashboards do we have?" mid-migration	Make Step 1 a hard gate
Auto-translating LOW silently	DT alert fires/doesn't fire when NR's equivalent did the opposite	Set translator threshold; require manual triage

Pitfall	Symptom	Mitigation
Cutting over alerts without dual-alert window	On-call surprises in week 2	Make 1–2 week dual-alert non-negotiable
Migrating notification channels but not rotating secrets	Migrated webhooks fail silently	Document secret rotation as part of Wave 3
Trying to make DT behave like NR	Endless customization debt	Adopt DT patterns (Workflows, OpenPipeline, Dynatrace Intelligence)
Single host group everywhere	Can't set per-environment thresholds	See FAQ-01 — plan host grouping in Step 3
Default bucket as permanent destination	Compliance + cost + IAM all suffer	See ORGNZ-02 / ORGNZ-99 — plan buckets in Step 3
Skipping SLO 7-day delta check	Compliance reporting drifts post-cutover	Compute the 7-day NR-vs-DT SLI delta by hand before declaring Wave 4 done; <code>audit-slos</code> checks metric validity only
Treating <code>migrate --diff</code> as a preview	Artifacts imported during what was meant to be a dry look	Preview with <code>migrate --dry-run --diff</code> ; <code>--diff</code> alone imports first
Partial rollback with <code>-rollback ... --components</code>	Every entity in the manifest deleted	<code>--components</code> is ignored by rollback; roll back from a filtered copy of the manifest, <code>--dry-run first</code>

4. Where the Deep Dives Live

NR2DT is the procedural runbook. NRLC is the deep-dive reference. Each NRLC notebook is standalone — you can read just the one you need.

When you want depth on...	Read
Why DT is structurally different from NR	NRLC-01 Platform Comparison
The NRQL → DQL compiler internals	NRLC-02 NRQL → DQL Translation
Dashboard widget translation, multi-page, variables	NRLC-03 Dashboard Migration

When you want depth on...	Read
Workflows event routing (replaces Alerting Profiles)	NRLC-04 Alert & Workflow Migration
Synthetic type mapping; scripted browser caveat	NRLC-05 Synthetic Migration
SLO math equivalence; OpenPipeline workload enrichment	NRLC-06 SLO & Workload Migration
OpenPipeline filter / parsing / tagging	NRLC-07 Logs, Tags & Drops
Three-tier validation; rollback manifest mechanics	NRLC-08 Validation, Diff & Rollback
migrate.py CLI and toolchain reference	NRLC-09 Toolchain Reference

5. When to Reuse This Series

This 9-step framework applies whenever you're migrating to Dynatrace from another observability platform with similar entity classes:

- New Relic → Dynatrace (this series)
- Datadog → Dynatrace (substitute Datadog query language and entity model)
- Splunk → Dynatrace (overlap with the **S2D** series)
- AppDynamics → Dynatrace (similar APM → OneAgent translation)

The procedural shape (discover → strategize → design → translate → wave migrate → validate → cutover) is platform-agnostic. The deep-dive content lives in the platform-specific deep-dive series (NRLC for New Relic, S2D for Splunk, etc.).

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).